19 · RxJS Fundamentals

Intermediate Angular learning notes

a. Definition

RxJS is a library for working with **streams of values over time**. An **Observable** describes a stream; nothing happens until you **subscribe**. **Operators** (`map`, `filter`, `take`…) are chained inside `pipe()` to transform the stream.

In templates, the `async` pipe subscribes for you and unsubscribes automatically, so you avoid memory leaks. A **Subject** is an Observable you can push values into manually (useful for events).

b. Diagram



Values flow through operators to the subscriber

c. Syntax

```
source$.pipe(
  filter(n => n % 2 === 0),
  map(n => n * 10),
).subscribe(value => console.log(value));

{{ source$ | async }}      // template: auto subscribe/unsubscribe
```

d. Example Program

Programs/19_RxJS_Fundamentals/rxjs-basics.ts

```
// Run with: npx tsx rxjs-basics.ts
import { filter, interval, lastValueFrom, map, Observable, of, Subject, take, tap } from
'rxjs';

async function main() {
  // 1. Creation + operators in pipe()
  of(1, 2, 3, 4, 5, 6)
    .pipe(
      filter((n) => n % 2 === 0), // keep even numbers
      map((n) => n * 10), // multiply by 10
    )
    .subscribe((v) => console.log('even x10:', v));

  // 2. Build your own Observable (note the cleanup function)
  const greeting$ = new Observable<string>((subscriber) => {
    subscriber.next('Hello');
    subscriber.next('Angular');
    subscriber.complete();
    return () => console.log('cleanup done');
  });
  greeting$.subscribe({
    next: (v) => console.log('got:', v),
```

```
    complete: () => console.log('stream completed'),
  });

  // 3. Subject: push values manually; every subscriber receives them
  const bell$ = new Subject<string>();
  bell$.subscribe((m) => console.log('Student A heard:', m));
  bell$.subscribe((m) => console.log('Student B heard:', m));
  bell$.next('Class over!');

  // 4. Timed stream: 3 ticks, 50 ms apart, then it completes
  const last = await lastValueFrom(
    interval(50).pipe(
      take(3),
      map((i) => i + 1),
      tap((n) => console.log('tick', n)),
    ),
  );
  console.log('last tick was', last);
}

main();
```

More programs in the same folder: `ticker.component.ts`

e. Output

Console output (run with `npx tsx`):

```
even x10: 20
even x10: 40
even x10: 60
got: Hello
got: Angular
stream completed
cleanup done
Student A heard: Class over!
Student B heard: Class over!
tick 1
tick 2
tick 3
last tick was 3
```

f. Real-World Example in Tamil (தமிழ் உதாரணம்)

சன் டிவி நேரலை ஒளிபரப்பு எப்போதும் ஓடிக்கொண்டே இருக்கிறது. நீங்கள் டிவியை ஆன் செய்தால்தான் (**subscribe**) அதைப் பார்க்க முடியும்; ஆஃப் செய்தால் (**unsubscribe**) உங்களுக்கு வருவது நின்றுவிடும். ஒளிபரப்பு = **Observable**.

விளம்பர இடைவேளையை நீக்கிப் பார்க்க விரும்பினால் அது `filter`; ஒவ்வொரு காட்சியையும் தமிழில் மாற்றிப் பார்த்தால் அது `map`. இவை எல்லாம் ஒளிபரப்பு வரும் வழியிலேயே (`pipe`) நடக்கும்.

g. Common Mistakes

- Forgetting to `subscribe`; nothing runs.

- Subscribing manually in components and never unsubscribing (memory leaks). Prefer `async` pipe, `toSignal` or `takeUntilDestroyed`.
- Nesting `subscribe` inside `subscribe`; use `switchMap` and friends instead.
- Confusing `of(1,2,3)` (three emissions) with `of([1,2,3])` (one array emission).

h. Interview Questions

Q1. What is an Observable?
A lazy stream of zero or more values over time that a consumer can subscribe to.

Q2. Observable vs Promise?
A Promise gives one value, is eager and cannot be cancelled; an Observable can emit many values, is lazy and cancellable.

Q3. What is a Subject?
An Observable that is also an Observer: you call `next()` on it to multicast values to all subscribers.

Q4. What does the async pipe do?
Subscribes to an Observable in the template, displays the latest value and unsubscribes on destroy.

Q5. What is an operator?
A function used in `pipe()` that transforms, filters or combines streams, e.g. `map`, `filter`, `take`.

i. Practice Exercises

1. Use `interval(1000)` with `take(10)` to build a countdown that prints 10 to 1.
2. From `of(1..20)` keep numbers divisible by 3 and square them.
3. Create a `Subject<string>` chat service and subscribe from two components.